

System Architecture Design

1. Architecture Objective

ระบบนี้ออกแบบเพื่อคำนวณราคาสุทธิของคำสั่งซื้อ โดยต้องรองรับ:

- Promotion หลายประเภท เช่น Percentage Discount, Fixed Amount Discount
- Promotion ระดับสินค้าและระดับ Order
- Promotion แบบซ้อนกัน
- การเพิ่ม Promotion ใหม่ในอนาคต
- การตรวจสอบย้อนหลังผ่าน Calculation Breakdown และ Audit Log
- การขยายระบบในระดับ Production

เป้าหมายหลักคือให้ระบบมีคุณสมบัติ:

Correct
Extensible
Maintainable
Testable
Auditable
Scalable

2. Recommended Technology Stack

Layer	Technology	Reason
Backend Language	Go	เร็ว, เหมาะกับ Backend Service, Concurrency ดี
HTTP Framework	Go Fiber	Lightweight, Routing ง่าย, Performance ดี
Database	MySQL	เหมาะกับ Relational Data เช่น Order, Product, Promotion
ORM	GORM	ใช้ Mapping Table, Association, Preload และ Transaction ได้สะดวก

Layer	Technology	Reason
Cache	Redis	Cache Active Promotions เพื่อลดภาระ Database
Logging	Zap / Zerolog	Structured Logging
Metrics	Prometheus	เก็บ metrics เช่น latency, applied promotion count
Tracing	OpenTelemetry	Trace request ตั้งแต่ API ถึง DB
Deployment	Docker / Kubernetes หรือ VM	รองรับ Horizontal Scaling

สำหรับ MVP สามารถเริ่มจาก:

```
Go Fiber + MySQL + GORM
```

แล้วค่อยเพิ่ม Redis, Metrics และ Tracing ใน Production Phase

3. High-Level Architecture

MVP Architecture

flowchart TD

```
C[Client / Frontend] --> API[Go Fiber API]
```

```
API --> PS[Pricing Service]
```

```
PS --> PR[Product Repository]
```

```
PS --> PE[Promotion Engine]
```

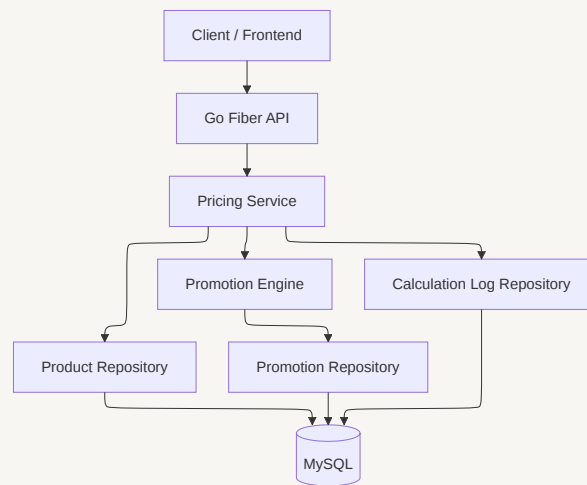
```
PS --> LR[Calculation Log Repository]
```

```
PR --> DB[(MySQL)]
```

```
PE --> PROMO_REPO[Promotion Repository]
```

```
PROMO_REPO --> DB
```

```
LR --> DB
```



Production Architecture

flowchart TD

C[Client / Frontend] --> LB[Load Balancer]

LB --> API1[Go Fiber App Instance 1]

LB --> API2[Go Fiber App Instance 2]

LB --> API3[Go Fiber App Instance N]

API1 --> REDIS[(Redis Cache)]

API2 --> REDIS

API3 --> REDIS

API1 --> DBP[(MySQL Primary)]

API2 --> DBP

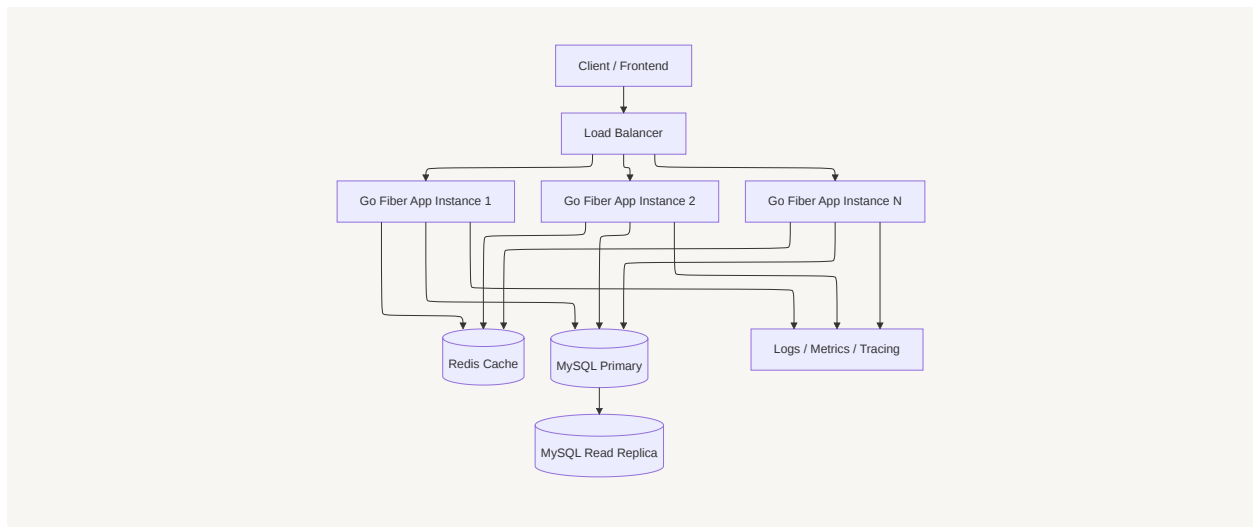
API3 --> DBP

DBP --> DBR[(MySQL Read Replica)]

API1 --> LOG[Logs / Metrics / Tracing]

API2 --> LOG

API3 --> LOG



4. Why Modular Monolith First

สำหรับโจทย์นี้ แนะนำเริ่มด้วย **Modular Monolith** ก่อน ไม่ควรรีบแยก Microservice ตั้งแต่แรก

เหตุผล:

- ระบบยังมี Domain ชัดเจนและขนาดไม่ใหญ่เกินไป
- พัฒนาและทดสอบง่ายกว่า
- Deploy ง่ายกว่า
- Transaction ระหว่าง Order และ Promotion Usage จัดการง่ายกว่า
- แยก Module ภายในไว้ชัดเจน ทำให้อนาคตแยกเป็น Service ได้

แนวคิด:

เริ่มเป็น Go Fiber App เดี่ยว
แต่ภายในแยก Layer ชัดเจน:
Handler → Service → Engine → Strategy → Repository

อนาคตถ้า Traffic สูงมาก สามารถแยก:

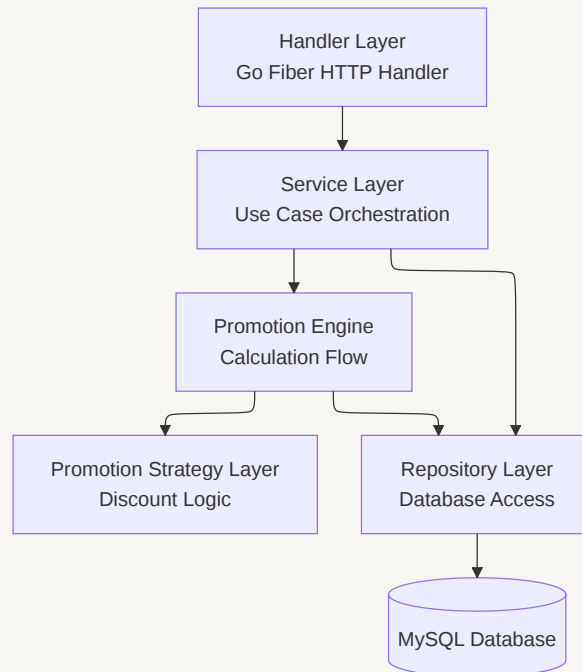
Promotion Engine → Promotion Service
Product Repository → Product Service

5. Layered Architecture

flowchart TD

H[Handler Layer
Go Fiber HTTP Handler]
S[Service Layer
Use Case Orchestration]
E[Promotion Engine
Calculation Flow]
P[Promotion Strategy Layer
Discount Logic]
R[Repository Layer
Database Access]
D[(MySQL Database)]

H --> S
S --> E
E --> P
S --> R
E --> R
R --> D



6. Layer Responsibilities

Layer	Responsibility	ห้ามทำ
Handler	รับ Request, Validate เบื้องต้น, Return Response	ห้ามคำนวณ Promotion
Service	คุม Use Case เช่น Calculate, Confirm Order	ห้ามเขียน SQL ตรง
Promotion Engine	คุม Flow การคำนวณ Promotion	ห้ามผูกกับ Fiber
Strategy	คำนวณ Discount แต่ละประเภท	ห้าม Query Database
Repository	Query Database ผ่าน GORM	ห้ามมี Business Logic หนัก
Model	Map กับ Database Table	ห้ามผูกกับ API Response
DTO	Request / Response Structure	ห้ามใช้แทน DB Model ตรงๆ

7. Core Components

7.1 Go Fiber API Layer

รับ HTTP Request จาก Client

API หลัก:

```
POST /orders/calculate
POST /orders/confirm
POST /promotions
GET /promotions
GET /calculation-logs/:id
```

หน้าที่:

- Parse request
- Validate input เบื้องต้น

- เพิ่ม request_id
 - เรียก Service
 - Return response
-

7.2 Pricing Service

เป็น Use Case หลักสำหรับคำนวณราคา

หน้าที่:

1. รับ Calculate Order Request
2. Validate business input
3. โหลด Product จาก Product Repository
4. สร้าง Calculation Context
5. เรียก Promotion Engine
6. Save Calculation Log
7. Return Calculation Result

Flow:

```
OrderHandler  
→ PricingService  
→ ProductRepository  
→ PromotionEngine  
→ CalculationLogRepository  
→ Response
```

7.3 Promotion Engine

หัวใจของระบบ

หน้าที่:

- โหลด Active Promotions
- ตรวจสอบ Condition

- ตรวจ Target
- Sort Promotion
- Apply Promotion ตาม Priority
- ตรวจ Stacking Policy
- Cap Discount
- สร้าง Applied/Skipped Breakdown

ภายใน Promotion Engine ควรมี Component:

```
Promotion Engine
├─ Calculation Context
├─ Eligibility Evaluator
├─ Promotion Sorter
├─ Stacking Policy
├─ Strategy Factory
└─ Promotion Strategies
```

7.4 Condition Evaluator

ใช้ตรวจว่า Promotion ใช้กับ Order นี้ได้หรือไม่

ตัวอย่าง Condition:

- Product ID ตรงหรือไม่
- Category ID ตรงหรือไม่
- Order Amount ถึงขั้นต่ำหรือไม่
- Coupon Code ถูกต้องหรือไม่
- Promotion ยังอยู่ในช่วงเวลาหรือไม่

ควรใช้แนวคิด **Specification Pattern**

```
Condition = ใช้ได้ไหม
Action = ลดเท่าไร
```

สองเรื่องนี้ควรแยกกัน

7.5 Promotion Sorter

ใช้เรียง Promotion ให้ผลลัพธ์เป็น Deterministic

Sort Rule:

```
scope_order ASC
priority ASC
created_at ASC
promotion_id ASC
```

Scope Order:

```
ITEM = 1
CART = 2
COUPON = 3
SHIPPING = 4
```

7.6 Stacking Policy

ใช้ควบคุม Promotion ซ้อนกัน

ตรวจ Field:

```
stackable
exclusive
stop_processing
priority
scope
conflict_group
```

Policy ตัวอย่าง:

- ถ้า `stackable = true` ใช้ร่วมกันได้
- ถ้า `stackable = false` และมี Promotion ใน Scope เดียวกันถูกใช้แล้ว ให้ Skip
- ถ้า `exclusive = true` หลัง Apply แล้วให้หยุด Promotion ที่ขัดแย้ง
- ถ้า `stop_processing = true` ให้หยุด Rule ถัดไปตาม Scope

7.7 Strategy Factory

ใช้เลือก Strategy ตาม `action_type`

ตัวอย่าง:

```
PERCENTAGE_DISCOUNT → PercentageDiscountStrategy
FIXED_AMOUNT_DISCOUNT → FixedAmountDiscountStrategy
CART_FIXED_DISCOUNT → CartFixedDiscountStrategy
BUY_X_GET_Y → BuyXGetYStrategy
FREE_SHIPPING → FreeShippingStrategy
```

ข้อดี:

เพิ่ม Promotion ใหม่โดยไม่แก้ Calculation Flow หลัก

7.8 Promotion Strategies

ใช้คำนวณ Discount แต่ละประเภท

ตัวอย่าง Strategy:

Strategy	หน้าที่
PercentageDiscountStrategy	ลดเป็นเปอร์เซ็นต์
FixedAmountDiscountStrategy	ลดเป็นจำนวนเงิน
CartPercentageDiscountStrategy	ลดทั้ง Cart เป็นเปอร์เซ็นต์
CartFixedDiscountStrategy	ลดทั้ง Cart เป็นจำนวนเงิน
BuyXGetYStrategy	ซื้อ X แถม Y
FreeShippingStrategy	ลดค่าส่ง

7.9 Repository Layer

ใช้ติดต่อ Database ผ่าน GORM

Repository หลัก:

Repository	Method สำคัญ
ProductRepository	FindByIDs
PromotionRepository	FindActivePromotions
OrderRepository	CreateOrderWithItems
CalculationLogRepository	Create
PromotionUsageRepository	LockAndConsumeUsage

ข้อสำคัญ:

ห้าม Query Database ใน Loop ของการคำนวณ

8. Internal Project Structure

โครงสร้าง Project ที่แนะนำ:

```

promotion-engine/
├── cmd/
│   └── server/
│       └── main.go
├── internal/
│   ├── config/
│   │   └── config.go
│   ├── database/
│   │   └── mysql.go
│   ├── handler/
│   │   ├── order_handler.go
│   │   └── promotion_handler.go
│   ├── service/
│   │   ├── pricing_service.go
│   │   ├── order_service.go
│   │   └── promotion_admin_service.go
│   └── promotion/
│       ├── engine.go
│       ├── context.go
│       ├── eligibility.go
│       ├── sorter.go
│       ├── stacking_policy.go
│       └── strategy/
│           ├── strategy.go
│           ├── factory.go
│           └── percentage_discount.go

```

```

├── fixed_amount_discount.go
├── repository/
│   ├── product_repository.go
│   ├── promotion_repository.go
│   ├── order_repository.go
│   └── calculation_log_repository.go
├── model/
│   ├── product.go
│   ├── order.go
│   ├── order_item.go
│   ├── promotion.go
│   ├── promotion_condition.go
│   ├── promotion_action.go
│   ├── promotion_target.go
│   └── promotion_calculation_log.go
├── dto/
│   ├── calculate_order_request.go
│   └── calculate_order_response.go
├── middleware/
│   ├── request_id.go
│   ├── logger.go
│   └── recover.go
├── go.mod
└── README.md

```

9. Runtime Flow: Calculate Order

sequenceDiagram

```

participant C as Client
participant H as Go Fiber Handler
participant S as Pricing Service
participant PR as Product Repository
participant E as Promotion Engine
participant DB as MySQL
participant L as Calculation Log Repository

```

C->>H: POST /orders/calculate

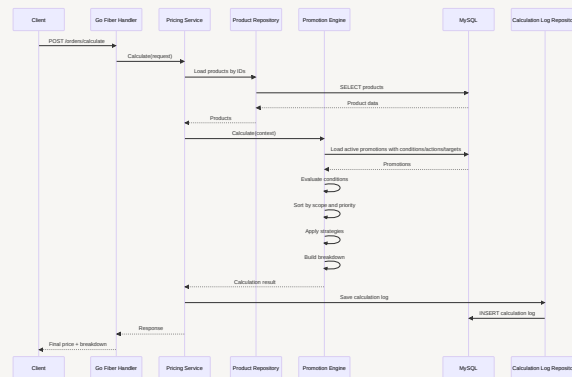
H->>S: Calculate(request)

S->>PR: Load products by IDs

PR->>DB: SELECT products

DB-->>PR: Product data

PR-->>S: Products
 S->>E: Calculate(context)
 E->>DB: Load active promotions with conditions/actions/targets
 DB-->>E: Promotions
 E->>E: Evaluate conditions
 E->>E: Sort by scope and priority
 E->>E: Apply strategies
 E->>E: Build breakdown
 E-->>S: Calculation result
 S->>L: Save calculation log
 L->>DB: INSERT calculation log
 S-->>H: Response
 H-->>C: Final price + breakdown



10. Runtime Flow: Confirm Order

sequenceDiagram

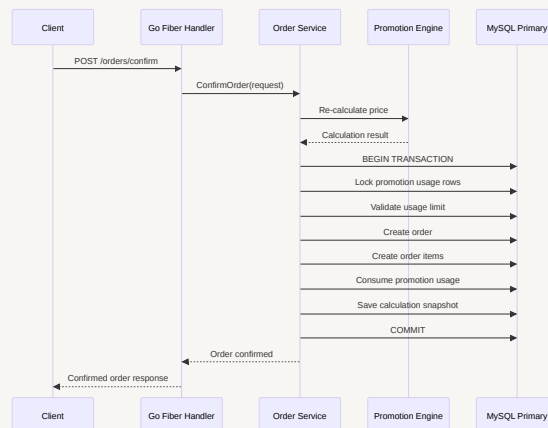
participant C as Client
 participant H as Go Fiber Handler
 participant S as Order Service
 participant E as Promotion Engine
 participant DB as MySQL Primary

C->>H: POST /orders/confirm

```

H->>S: ConfirmOrder(request)
S->>E: Re-calculate price
E-->>S: Calculation result
S->>DB: BEGIN TRANSACTION
S->>DB: Lock promotion usage rows
S->>DB: Validate usage limit
S->>DB: Create order
S->>DB: Create order items
S->>DB: Consume promotion usage
S->>DB: Save calculation snapshot
S->>DB: COMMIT
S-->>H: Order confirmed
H-->>C: Confirmed order response

```



เหตุผลที่ Confirm ต้อง Re-calculate:

- Promotion อาจหมดอายุ
- Promotion อาจถูกปิด
- Coupon usage อาจเต็ม
- Client อาจแก้ Payload
- ราคาสินค้าอาจเปลี่ยน

11. Database Architecture

ใช้ MySQL เป็น Primary Database

กลุ่ม Table หลัก:

```
Product Data
- products
- product_categories

Order Data
- orders
- order_items

Promotion Config
- promotions
- promotion_conditions
- promotion_actions
- promotion_targets

Runtime / Audit
- promotion_usages
- promotion_calculation_logs
```

แนวคิดสำคัญ:

```
Promotion ไม่ควร hardcode ใน Code
Promotion ต้องเก็บเป็น Config ใน Database
```

12. Cache Architecture

สำหรับ Production ควรใช้ Redis Cache สำหรับ Active Promotions

เหตุผล:

```
Promotion เป็นข้อมูล Read-heavy และเปลี่ยนไม่บ่อย
```

Cache สิ่งเหล่านี้:

- active promotions
- conditions

- actions
- targets

Cache Key ตัวอย่าง:

```
active_promotions:global
active_promotions:{sales_channel}
active_promotions:{sales_channel}:{user_segment}
```

TTL:

30 - 300 seconds

เมื่อ Promotion ถูก Create/Update/Delete:

Invalidate active promotion cache

ข้อสำคัญ:

Cache ใช้กับ Calculate ได้
แต่ Confirm Order ต้องใช้ Primary Database เมื่อต้อง Consume Usage

13. Database Access Pattern

Query ที่ควรเกิดใน `/orders/calculate`

1. Load products by product_ids
2. Load active promotions with conditions/actions/targets
3. Save calculation log

สิ่งที่ห้ามทำ:

```
for each item:
    query promotions
```



```
for each promotion:
    query conditions
```

เพราะจะเกิด N+1 Query Problem

ควรใช้ GORM Preload:

```
Preload Conditions
Preload Actions
Preload Targets
```

แล้วคำนวณใน Memory

14. Data Structure in Promotion Engine

หลังโหลด Promotion มาแล้ว ควรแปลงเป็น Map เพื่อลดการ Scan

```
promotionsByScope
productPromotionMap
categoryPromotionMap
stoppedScopes
appliedPromotions
skippedPromotions
```

ตัวอย่าง:

```
productPromotionMap:
product_id 1 → [Promo A, Promo B]
product_id 2 → [Promo C]

categoryPromotionMap:
category_id 10 → [Promo D]
```

15. Observability Architecture

ระบบควรมี:

Component	Purpose
Request ID	Trace request
Structured Log	Debug เหตุการณ์
Calculation Log	Audit ราคา
Metrics	Monitor performance
Tracing	ดู flow ข้าม component

Log Event สำคัญ:

```
calculation_started
calculation_completed
promotion_applied
promotion_skipped
promotion_invalid_config
promotion_strategy_not_found
promotion_usage_consumed
calculation_failed
```

Metrics สำคัญ:

```
promotion_calculation_latency_ms
promotion_applied_total
promotion_skipped_total
promotion_strategy_not_found_total
promotion_cache_hit_total
promotion_cache_miss_total
promotion_calculation_failed_total
```

16. Security Architecture

หลักสำคัญ:

- ห้ามเชื่อราคาจาก Client
- Server ต้องโหลด Product Price เอง
- `user_id` ใน Production ควรมาจาก Auth Context
- Admin API ต้อง Protected

- Coupon Code ต้อง Validate ฝั่ง Server
 - Promotion Config ต้อง Validate ก่อน Save
 - Confirm Order ต้อง Re-calculate
-

17. Scalability Design

MVP Scale

```
Go Fiber App
MySQL
GORM Preload
No DB Query in Loop
```

Medium Scale

```
Go Fiber หลาย Instance
Redis Cache
MySQL Read Replica
Connection Pool
Structured Logging
Metrics
```

High Scale

```
Load Balancer
App Instances หลายตัว
Redis Cluster
MySQL Primary + Read Replica
Async Log Queue
Partition Calculation Logs
Separate Promotion Service ถ้าจำเป็น
```

18. Architecture Decision Records

ADR-001: ใช้ Modular Monolith ก่อน

เหตุผล:

- พัฒนาเร็ว
 - Test ง่าย
 - Transaction ง่าย
 - แยก Module ชัด
 - พร้อมแยก Service ในอนาคต
-

ADR-002: Promotion Engine ไม่ผูกกับ Fiber หรือ GORM

เหตุผล:

- Unit Test ง่าย
 - Business Logic ไม่ผูกกับ Framework
 - เปลี่ยน Database หรือ Transport Layer ได้ง่าย
-

ADR-003: ใช้ Strategy Pattern สำหรับ Promotion Type

เหตุผล:

- เพิ่ม Promotion ใหม่ได้ง่าย
 - ลด if-else
 - แยก Test แต่ละ Promotion ได้
-

ADR-004: ใช้ Rule-based Database Schema

เหตุผล:

- เพิ่ม Promotion ใหม่ผ่าน Config ได้
 - แยก Condition / Target / Action ชัดเจน
 - ไม่ต้องสร้าง Table ใหม่ทุก Promotion Type
-

ADR-005: ใช้ Redis Cache สำหรับ Active Promotions ใน Production

เหตุผล:

- ลด Load บน MySQL
- ลด Latency
- เหมาะกับข้อมูลที่อ่านบ่อยและเปลี่ยนไม่บ่อย

19. Final System Architecture Summary

Architecture ที่เหมาะสมคือ:

```
Client
→ Go Fiber API
→ Pricing Service
→ Promotion Engine
  → Condition Evaluator
  → Promotion Sorter
  → Stacking Policy
  → Strategy Factory
  → Promotion Strategies
→ Repository Layer
→ MySQL
→ Redis Cache for Active Promotions
→ Calculation Audit Log
```

จุดแข็งของ Design นี้:

- คำนวณ Promotion ได้ถูกต้อง
- รองรับ Promotion ซ้อนกัน
- เพิ่ม Promotion ใหม่ได้โดยไม่กระทบ Flow หลัก
- Database ยืดหยุ่น
- Test ง่าย
- Debug และ Audit ได้
- Scale ได้ด้วย Cache, Index และ Stateless App

สรุปสั้นสำหรับใส่สไลด์

ระบบออกแบบเป็น Modular Monolith ด้วย Go Fiber, MySQL และ GORM โดยแยก Handler, Service, Promotion Engine, Strategy และ Repository ออกจากกัน Promotion Engine ทำหน้าที่คุม Calculation Flow ส่วน Logic ของ Promotion แต่ละประเภทแยกอยู่ใน Strategy ทำให้เพิ่ม Promotion ใหม่ได้โดยไม่กระทบ Flow หลัก ระบบรองรับ Audit Log, Redis Cache และ Transaction สำหรับ Confirm Order เพื่อให้พร้อมต่อยอดสู่ Production.